A Missing Proofs

Proof of Theorem 1. We prove that PipeShapley satisfies PAE, PAS, PANP, and LIN.

PAE:

$$\begin{aligned}
&\sum_{i,j} \phi_{i,j}^{PSV}(O_+, \mathcal{M}, u) \\
=&\sum_{i,j} \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (u(P_{\geq j}) - u(P_{>j})) \right] \\
=&\sum_{j} \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[u(P_{\geq j}) - u(P_{>j}) \right] \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[u(P_{\geq 1}) - u(P_{>1}) + u(P_{\geq 2}) - u(P_{>2}) + \cdots + u(P_{\geq \overline{M}}) - u(P_{>\overline{M}}) \right] \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[u(P) - u(P_{\geq 2}) + u(P_{\geq 2}) - u(P_{\geq 3}) + \cdots + u(P_{\geq \overline{M}}) - u(\emptyset) \right] \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[u(P) - u(\emptyset) \right].
\end{aligned}$$

Therefore, PipeShapley satisfies position-aware efficiency.

PAS: If $u(P_{p_j = o_i}) = u(P_{p_j = o_{i'}})$ for any pipeline $P \in O_+^{\overline{M}}$, then we have

$$\begin{aligned}
&\phi_{i,j}^{PSV}(O_+, \mathcal{M}, u) = \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (u(P_{\geq j}) - u(P_{>j})) \right] \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_{i'}] \cdot (u(P_{\geq j}) - u(P_{>j})) \right] \\
=&\phi_{i',j}^{PSV}(O_+, \mathcal{M}, u).
\end{aligned}$$

Therefore, PipeShapley satisfies position-aware symmetry.

PANP: If $u(P) = u(P_{-j})$ for any pipeline $P \in O_+^{\overline{M}}$ with $p_j = o_i$, then we have $u(P_{\geq j}) = u(P_{>j})$. Then, we have

$$\phi_{i,j}^{PSV}(O_+, \mathcal{M}, u) = \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (u(P_{\geq j}) - u(P_{>j})) \right] = 0.$$

□

Therefore, PipeShapley satisfies position-aware null player.

LIN: For any utility metrics u_1, u_2 and any weights $w_1, w_2 \in R$, we have

$$\begin{aligned}
&\phi_{i,j}^{PSV}(O_+, \mathcal{M}, w_1 \cdot u_1 + w_2 \cdot u_2) \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot ((w_1 \cdot u_1 + w_2 \cdot u_2)(P_{\geq j}) - (w_1 \cdot u_1 + w_2 \cdot u_2)(P_{>j})) \right] \\
=&\mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (w_1 \cdot u_1(P_{\geq j}) - w_1 \cdot u_1(P_{>j})) \right] + \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (w_2 \cdot u_2(P_{\geq j}) - w_2 \cdot u_2(P_{>j})) \right] \\
=&w_1 \cdot \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (u_1(P_{\geq j}) - u_1(P_{>j})) \right] + w_2 \cdot \mathbb{E}_{P \in O_+^{\overline{M}} | \prec\prec_{O_+}} \left[\mathbf{1}[p_j = o_i] \cdot (u_2(P_{\geq j}) - u_2(P_{>j})) \right] \\
=&w_1 \cdot \phi_{i,j}^{PSV}(O_+, \mathcal{M}, u_1) + w_2 \cdot \phi_{i,j}^{PSV}(O_+, \mathcal{M}, u_2).
\end{aligned}$$

Therefore, PipeShapley satisfies linearity.

B Additional Scalability Analyses

This section provides supporting evidence for the large-dataset experiments in the main paper. We study: (1) the fidelity of 10K-subsample pipeline search, (2) parallel wall-clock scalability, and (3) the separation between one-time offline cost and per-dataset online cost.

B.1 Fidelity of Subsampled Pipeline Search

During pipeline construction, H-SP estimates pipeline utility using a uniformly sampled subset of 10,000 training records. The selected pipeline is subsequently retrained and evaluated on the full dataset, and only full-data accuracy is reported. Because PipeShapley is used to rank candidate operators rather than to predict exact final accuracy, this approximation is effective when the subsample preserves the relative operator ordering.

For each pipeline position, we compute the Spearman correlation between the operator-level PipeShapley rankings obtained using the full training data and the 10K subset. We use the same sampled pipeline contexts and random seeds in both settings.

Table 1 shows that the 10K-subsample protocol reduces the average search time from 17,220 seconds to 3,650 seconds, a 4.71× speedup, while preserving the same average full-data accuracy of 0.705.

Table 1: Fidelity of 10K-subsample pipeline search. ρ is the Spearman correlation between operator rankings estimated using the full training set and the 10K subset. ΔAcc is the accuracy of H-SP minus that of full-data H-SP.

Dataset	Full-data H-SP		H-SP		ΔAcc	Spearman ρ	Speedup
	Acc.	Time (s)	Acc.	Time (s)			
HIGGS	0.706	17,488	0.706	3,593	+0.000	0.611	4.87×
Covertype	0.768	19,004	0.761	3,825	−0.007	0.656	4.97×
Airlines	0.641	15,169	0.647	3,533	+0.006	0.620	4.29×
Average	0.705	17,220	0.705	3,650	0.000	0.629	4.71×

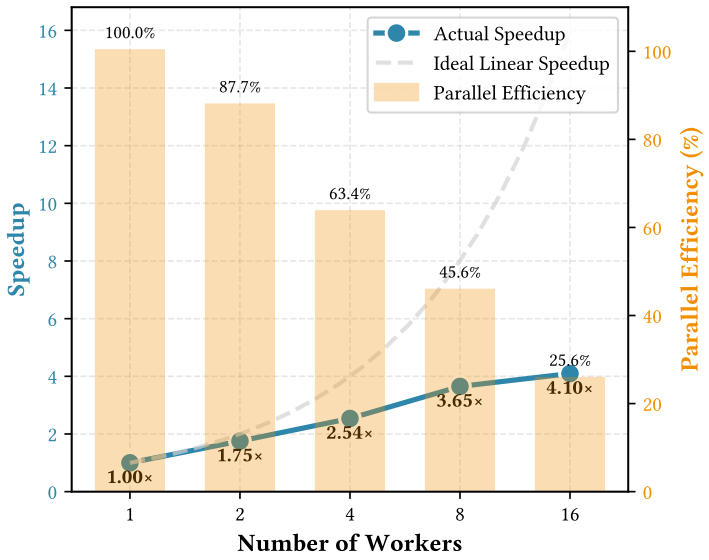


Figure 1: Parallel scalability of H-SP averaged over HIGGS, Covertype, and Airlines. Actual speedup is computed relative to the one-worker runtime. Parallel efficiency is speedup divided by the number of workers. The dashed curve denotes ideal linear speedup.

B.2 Parallel Wall-Clock Scalability

Within a fixed pipeline position, the utility evaluations used for PipeShapley estimation are mutually independent and can therefore be distributed across workers. Pipeline positions remain sequential because the operator selected at position j conditions the search at position $j + 1$. We therefore report wall-clock runtime, speedup, and parallel efficiency for 1, 2, 4, 8, and 16 workers.

As shown in Figure 1, H-SP achieves consistent wall-clock acceleration as the number of workers increases. The 16-worker setting reduces the average online construction time from 3,650s to approximately 891s, yielding a 4.10× speedup and 25.6% parallel efficiency. The speedup is sublinear because pipeline positions are constructed sequentially, while worker initialization, data transfer, model fitting, and synchronization introduce additional overhead.

B.3 Offline and Online Cost Accounting

The methods in the large-dataset comparison operate under different deployment regimes. CtxPipe and HAIPipe first train an RL policy offline and then use the trained policy for online inference. Learn2Clean does not require separate offline pretraining; its reported runtime includes online training and inference. H-SP uses a lightweight MAB warm start based on $T_{\text{mab}} = 1000$ historical pipeline evaluations.

For an offline component reused across Q target datasets, the amortized wall-clock cost per target dataset is

$$T_{\text{amortized}}(Q) = \frac{T_{\text{offline}} + Q\,T_{\text{online}}}{Q}. \tag{1}$$

This formulation distinguishes the cold-start regime from the high-reuse regime, in which a pretrained policy is amortized over many target datasets.

Table 2: Offline and online wall-clock cost. Online time is averaged over HIGGS, Covertype, and Airlines. The cold-start total is the sum of the one-time offline component and one target-dataset run.

Method	Offline cost	Online time	Cold-start total	Offline component	Reuse scope / reported-time semantics
CtxPipe	≈ 20 h	311 s	≈ 20.1 h	RL policy training	Policy reused after training; the main-paper table reports online inference only.
HAIPipe	≈ 10 h	309 s	≈ 10.1 h	RL policy training	Policy reused after training; the main-paper table reports online inference only.
Learn2Clean	None	1,246 s	1,246 s	–	No separate pretraining; reported time includes online training and inference.
H-SP	1,288 s	3,650 s	4,938 s	MAB warm start	Reusable across target datasets sharing the operator library and downstream learner; reported time is online construction only.
H-SP$_{16W}$	1,288 s	891 s	2,179 s	MAB warm start	Same warm start as H-SP; reported time is 16-worker online construction.

C Datasets, Constraints, and Additional Results

C.1 Dataset Details

We conduct experiments on 27 real-world classification datasets collected from OpenML, including 18 benchmark datasets adopted by DiffPrep, 9 additional datasets from the CC18 suite for extended evaluation. With respect to scalability tests, we also include another 3 large datasets. These datasets cover a wide variety of domains including biology, text processing, healthcare, social science and software engineering. The datasets exhibit substantial diversity in terms of dataset scale and label cardinality. The details are shown at Table 3.

Table 3: Dataset characteristics. #Inst.: number of instances; #Feat.: number of features; #Lbl.: number of class labels; #Miss.: number of missing values; #Out.: number of outliers.

	Data Characteristics					
Dataset	**Domain**	**#Inst.**	**#Feat.**	**#Lbl.**	**#Miss.**	**#Out.**
abalone	biology	4,177	9	28	0	200
ada_prior	machine learning	4,562	15	2	88	423
avila	pattern recognition	20,867	11	12	0	4,458
connect-4	games & AI	67,557	43	3	0	45,873
eeg	healthcare	14,980	15	2	0	209
google	web	9,367	9	2	1,639	109
house_prices	socioeconomic	1,460	81	2	6,965	617
jungle_chess	games&AI	44,819	7	3	0	0
micro	biomedical	20,000	21	5	0	8,122
mozilla4	web	15,545	6	2	0	290
obesity	healthcare	2,111	17	7	0	25
page-blocks	document analysis	5,473	11	5	0	1,011
pbcseq	healthcare	1,945	19	2	1,445	99
pol	remote sensing	15,000	49	2	0	8,754
run_or_walk	sensor	88,588	7	2	0	8,548
shuttle	space technology	58,000	10	7	0	5,341
uscensus	socioeconomic	32,561	15	2	4,262	2,812
wall-robot	robotics&sensor	5,456	25	4	0	1,871
cmc	biology	1,473	9	3	0	127
diabetes	healthcare	768	8	2	0	93
jm1	software	10,885	21	2	25	2,962
kc1	software	2,109	21	2	0	959
kc2	software	522	21	2	0	118
pc4	software	1,458	37	2	0	893
phoneme	sensor	5,404	5	2	0	151
qsar-biodeg	biology	1,055	41	2	0	761
vehicle	sensor	846	18	4	0	44
HIGGS	machine learning	98,050	28	2	9	9,042
Airlines	traffic	539,383	8	2	0	9,910
Covertype	nature/biology	581,012	55	7	0	55,564

C.2 Operator Level Constraints Details

In Section V-A in the main paper, we apply high-level partial order constraints:

$$\text{Imputation} \prec \text{Encoding} \prec \{\text{Scaling}, \text{Engineering}, \text{Selection}\},$$

which means missing value imputation occur before encoding, and encoding precede numerical feature operations.

Other than constraints explained above, there are operator level constraints that trims ineffective sequences. The details are listed below:

- Too many feature expansion operators (Polynomial Features, Interaction Features, Random Trees Embedding) applied to a dataset, then data features will increase from within 100 to over 1 million. In this case, the preprocessed data has too many redundant features, and may cause slow running speed or out of memory error.
- Successive dimensionality reduction operators are not allowed. Since these operations may compress data features while losing potential information, keep reducing data features may lead to over-transformation and failure to train and predict based on few data features.

C.3 Operator Repetition Cases

This subsection illustrates the 14 cases with repeated operators within single pipeline across the 27 dataset × 5 random seeds. Column SV Comparison shows the difference of PipeShapley Values of the repeated operators within the pipeline. Take the first row as an instance, the PipeShapley Value of operator PolynomialFeatures (15) is 0.0108 at position 3, but only 0.0004 at position 4.

Table 4: Cases of pipelines with operator repetitions. Operators use the IDs in Table II in the main paper, repeated occurrences are underlined. ΔAccuracy denotes the accuracy reduction after removing the repeated operations from the Repetitive Pipe.

Dataset	Seed	Pipe with repetition	Accuracy	SV Comparison	Pipe without repetition	Accuracy	ΔAccuracy
abalone	42	[4,9,15,15]	0.2766	0.0108/0.0004	[4,9,15]	0.2731	-0.35%
abalone	114	[4,15,13,15]	0.2766	0.0109/0.0004	[4,15,13]	0.2695	-0.71%
micro	114	[12,16,12]	0.6443	0.0168/0.0028	[12,16]	0.6415	-0.28%
pbcseq	114	[0,2,6,24,23,23]	0.7738	0.0502/0.0180	[0,2,6,24,23]	0.7738	0.00%
uscensus	114	[0,6,10,10]	0.8457	0.0116/0.0099	[0,6,10]	0.8457	0.00%
avila	256	[11,11,23]	0.9411	0.1192/0.0005	[11,23]	0.9406	-0.05%
uscensus	256	[0,6,10,10]	0.8457	0.0116/0.0099	[0,6,10]	0.8457	0.00%
avila	512	[11,11,23]	0.9411	0.1192/0.0005	[11,23]	0.9406	-0.05%
google	512	[0,1,5,12,13,12]	0.6708	0.0186/0.0069	[0,1,5,12,13]	0.6638	-0.70%
micro	512	[12,16,12]	0.6443	0.0168/0.0028	[12,16]	0.6415	-0.28%
obesity	512	[6,11,11,23]	0.9196	0.0024/0.0054	[6,11,23]	0.9173	-0.23%
uscensus	512	[0,6,10,10]	0.8457	0.0116/0.0099	[0,6,10]	0.8457	0.00%
avila	768	[11,11,23]	0.9411	0.1192/0.0005	[11,23]	0.9406	-0.05%
pbcseq	768	[0,1,6,12,23,23]	0.7763	0.0402/0.0129	[0,1,6,12,23]	0.7737	-0.26%

C.4 Visualization for Positional Shapley Values

Figure 2 shows all the positional shapley values of dataset `obesity`. The heatmap reveals that the same operator exhibits drastically different contributions at different positions. For instance, `QuantileTransformer` shows high contribution at position 2 ($\phi_{i,2} = 0.034$) but negative contribution at position 6 ($\phi_{i,6} = -0.017$), a pattern that traditional position-agnostic Shapley values fundamentally cannot capture.

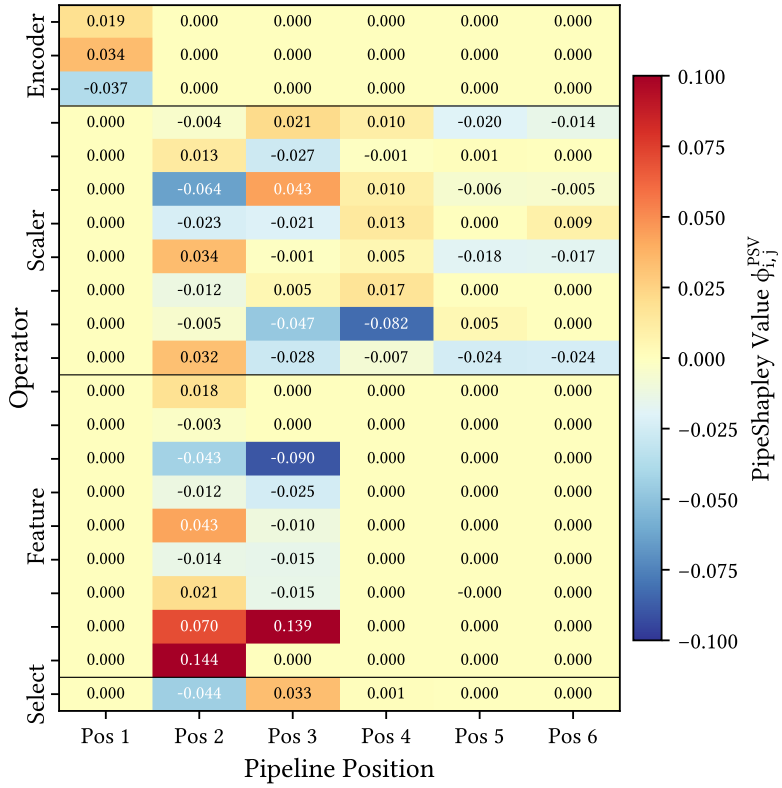


Figure 2: Visualization for shapley values on dataset `obesity`. It reveals that operators at different positions have variant shapley values, indicating different selection options at each slot.

C.5 Extended Results with Different Downstream Tasks

While the main paper focuses on Logistic Regression as a representative downstream model, the proposed pipeline construction framework is not tied to a specific learner. To demonstrate its robustness across different model families, we further evaluate our method using Random Forest, XGBoost, and Multi-Layer Perceptron (MLP) as downstream classifiers. This section reports the corresponding experimental results for these additional downstream models.

Table 5: Performance Comparison on Search Space ($M = 6, N = 25$) with Random Forest Classifier as the downstream task.

Dataset	Random	TPOT	AUSK	CtxPipe	HAIPipe	CatDB	ML-Agent	GPT4o-ZS	GPT4o-FS	LOO	MC-SP	H-SP
abalone	0.251	0.253	0.270	0.245	0.257	0.116	0.215	0.230	0.253	0.243	0.250	0.250
ada_prior	0.838	0.846	0.840	0.832	0.812	0.747	0.838	0.828	0.837	0.847	0.834	0.834
avila	0.598	0.924	0.742	0.810	0.670	0.419	0.906	0.790	0.881	0.995	0.994	0.993
connect-4	0.676	0.737	0.698	0.709	0.747	0.825	N/A	0.687	0.604	0.815	0.819	0.814
eeg	0.661	0.906	0.820	0.876	0.613	0.526	0.781	0.925	0.951	0.938	0.939	0.937
google	0.631	0.681	0.650	0.593	0.594	0.591	N/A	0.685	0.630	0.710	0.714	0.699
house_prices	0.916	0.908	0.917	0.904	0.908	0.925	0.939	0.877	0.880	0.928	0.928	0.938
jungle_chess	0.671	0.821	0.794	0.783	0.697	0.819	N/A	0.825	0.785	0.919	0.858	0.884
micro	0.584	0.607	0.591	0.558	0.625	0.611	0.785	0.624	0.620	0.618	0.620	0.620
mozilla4	0.922	0.944	0.946	0.935	0.871	0.953	0.946	0.916	0.944	0.957	0.950	0.958
obesity	0.850	0.967	0.897	0.842	0.825	0.936	0.956	0.879	0.894	0.967	0.964	0.953
page-blocks	0.965	0.968	0.974	0.970	0.963	0.975	0.984	0.976	0.969	0.975	0.975	0.974
pbcseq	0.734	0.807	0.747	0.789	0.735	0.789	0.816	0.746	0.746	0.789	0.789	0.817
pol	0.879	0.976	0.957	0.962	0.969	0.877	0.968	0.899	0.851	0.987	0.981	0.986
run_or_walk	0.833	0.979	0.980	0.991	0.875	0.989	0.796	0.927	0.980	0.991	0.975	0.990
shuttle	0.999	0.999	0.999	1.000	0.978	0.999	0.916	1.000	0.999	1.000	0.999	0.999
uscensus	0.846	0.848	0.853	0.839	0.832	0.845	0.845	0.829	0.846	0.847	0.847	0.847
wall-robot	0.882	0.998	0.985	0.949	0.875	0.990	0.997	0.925	0.937	0.995	0.968	0.995
cmc	0.554	0.547	0.482	0.508	0.549	0.516	0.590	0.507	0.507	0.547	0.588	0.534
diabetes	0.833	0.831	0.790	0.688	0.773	0.754	0.616	0.831	0.818	0.740	0.844	0.844
jm1	0.821	0.821	0.813	0.801	0.809	0.807	0.783	0.826	0.826	0.828	0.826	0.826
kc1	0.839	0.882	0.852	0.836	0.865	0.851	0.880	0.829	0.853	0.862	0.900	0.900
kc2	0.796	0.811	0.819	0.857	0.857	0.848	0.852	0.830	0.830	0.811	0.849	0.849
pc4	0.883	0.918	0.891	0.890	0.914	0.894	0.846	0.918	0.918	0.904	0.917	0.918
phoneme	0.757	0.902	0.799	0.810	0.772	0.908	0.923	0.904	0.893	0.911	0.915	0.917
qsar-biodeg	0.774	0.877	0.850	0.872	0.867	0.858	0.914	0.840	0.840	0.868	0.877	0.877
vehicle	0.702	0.800	0.754	0.747	0.812	0.724	0.747	0.812	0.812	0.835	0.870	0.871
AVERAGE	0.766	0.835	0.804	0.800	0.780	0.781	0.827	0.810	0.811	0.845	0.851	0.853

Table 6: Performance Comparison on Search Space ($M = 6, N = 25$) with XGBoosting Classifier as the downstream task.

Dataset	Random	TPOT	AUSK	CtxPipe	HAIPipe	CatDB	ML-Agent	GPT4o-ZS	GPT4o-FS	LOO	MC-SP	H-SP
abalone	0.255	0.283	0.245	0.246	0.260	0.266	0.215	0.231	0.229	0.226	0.259	0.267
ada_prior	0.812	0.852	0.848	0.825	0.855	0.747	0.844	0.831	0.831	0.813	0.847	0.855
avila	0.808	0.886	0.883	0.779	0.834	0.418	0.880	0.804	0.914	0.998	0.998	0.998
connect-4	0.676	0.739	0.735	0.735	0.758	0.767	N/A	0.745	0.656	0.838	0.819	0.849
eeg	0.631	0.810	0.855	0.757	0.832	0.526	0.752	0.935	0.951	0.958	0.965	0.886
google	0.631	0.669	0.685	0.595	0.684	0.638	N/A	0.670	0.670	0.649	0.695	0.692
house_prices	0.916	0.935	0.917	0.884	0.911	0.921	0.939	0.908	0.921	0.921	0.938	0.942
jungle_chess	0.671	0.824	0.948	0.842	0.802	0.826	N/A	0.856	0.820	0.979	0.982	0.979
micro	0.588	0.618	0.619	0.621	0.632	0.620	0.664	0.617	0.623	0.623	0.620	0.620
mozilla4	0.902	0.944	0.953	0.937	0.903	0.947	0.940	0.912	0.940	0.955	0.945	0.943
obesity	0.850	0.957	0.967	0.780	0.946	0.948	0.957	0.865	0.913	0.962	0.967	0.967
page-blocks	0.965	0.969	0.971	0.968	0.971	0.976	0.991	0.975	0.967	0.972	0.969	0.973
pbcseq	0.737	0.789	0.832	0.766	0.781	0.748	0.838	0.776	0.774	0.830	0.815	0.841
pol	0.879	0.968	0.981	0.946	0.972	0.800	0.980	0.934	0.908	0.985	0.986	0.984
run_or_walk	0.833	0.974	0.972	0.987	0.979	0.987	0.702	0.982	0.924	0.991	0.975	0.991
shuttle	0.992	0.998	0.999	0.999	0.999	0.998	0.888	1.000	0.999	1.000	0.999	0.999
uscensus	0.823	0.853	0.862	0.851	0.837	0.872	0.872	0.834	0.862	0.869	0.866	0.869
wall-robot	0.902	0.997	0.993	0.939	0.996	0.995	0.999	0.944	0.943	0.997	0.997	0.997
cmc	0.554	0.561	0.565	0.529	0.542	0.560	0.580	0.520	0.541	0.527	0.601	0.568
diabetes	0.833	0.831	0.831	0.734	0.766	0.779	0.616	0.818	0.818	0.792	0.831	0.857
jm1	0.821	0.813	0.804	0.807	0.804	0.804	0.761	0.817	0.817	0.804	0.827	0.816
kc1	0.839	0.863	0.838	0.851	0.855	0.846	0.895	0.853	0.863	0.863	0.882	0.882
kc2	0.796	0.811	0.811	0.829	0.848	0.857	0.840	0.811	0.887	0.849	0.849	0.868
pc4	0.883	0.925	0.901	0.901	0.914	0.904	0.961	0.945	0.945	0.932	0.932	0.925
phoneme	0.757	0.874	0.787	0.865	0.846	0.861	0.859	0.891	0.889	0.908	0.915	0.913
qsar-biodeg	0.767	0.887	0.782	0.863	0.872	0.867	0.888	0.821	0.821	0.840	0.887	0.887
vehicle	0.730	0.776	0.732	0.729	0.812	0.747	0.729	0.847	0.824	0.847	0.894	0.894
AVERAGE	0.772	0.830	0.827	0.799	0.823	0.786	0.816	0.820	0.824	0.849	0.861	0.862

Table 7: Performance Comparison on Search Space ($M = 6, N = 25$) with MLP Classifier as the downstream task.

Dataset	Random	TPOT	AUSK	CtxPipe	HAIPipe	CatDB	ML-Agent	GPT4o-ZS	GPT4o-FS	LOO	MC-SP	H-SP
abalone	0.249	0.272	0.272	0.291	0.249	0.132	0.269	0.275	0.242	0.243	0.285	0.292
ada_prior	0.830	0.847	0.842	0.830	0.826	0.682	0.829	0.844	0.841	0.815	0.857	0.843
avila	0.763	0.763	0.778	0.691	0.561	0.250	0.779	0.818	0.856	0.781	0.800	0.858
connect-4	0.669	0.728	0.785	0.750	0.669	0.836	N/A	0.661	0.661	0.804	0.816	0.804
eeg	0.832	0.878	0.905	0.838	0.730	0.474	0.576	0.934	0.946	0.901	0.885	0.933
google	0.614	0.617	0.653	0.615	0.600	0.554	N/A	0.658	0.643	0.635	0.677	0.675
house_prices	0.912	0.925	0.914	0.897	0.914	0.921	0.930	0.880	0.904	0.918	0.942	0.925
jungle_chess	0.715	0.809	0.837	0.866	0.715	0.890	N/A	0.870	0.797	0.861	0.860	0.863
micro	0.594	0.624	0.627	0.615	0.596	0.618	0.514	0.605	0.601	0.632	0.634	0.639
mozilla4	0.911	0.863	0.932	0.936	0.889	0.883	0.894	0.912	0.941	0.942	0.936	0.922
obesity	0.823	0.934	0.899	0.903	0.816	0.738	0.906	0.922	0.936	0.903	0.891	0.905
page-blocks	0.945	0.973	0.964	0.961	0.946	0.960	0.989	0.980	0.980	0.963	0.963	0.953
pbcseq	0.764	0.766	0.754	0.766	0.792	0.694	0.745	0.766	0.781	0.766	0.792	0.797
pol	0.933	0.968	0.977	0.941	0.806	0.950	0.967	0.933	0.952	0.976	0.978	0.979
run_or_walk	0.925	0.972	0.989	0.866	0.972	0.970	0.675	0.927	0.927	0.972	0.990	0.988
shuttle	0.997	0.997	0.999	0.999	0.919	0.999	0.976	0.999	0.999	0.998	0.999	0.987
uscensus	0.838	0.842	0.845	0.846	0.838	0.853	0.995	0.834	0.840	0.847	0.847	0.840
wall-robot	0.878	0.955	0.941	0.925	0.866	0.875	0.903	0.957	0.953	0.942	0.957	0.957
cmc	0.551	0.595	0.543	0.500	0.568	0.560	0.537	0.581	0.561	0.601	0.601	0.615
diabetes	0.729	0.844	0.764	0.779	0.818	0.779	0.626	0.792	0.766	0.831	0.818	0.831
jm1	0.793	0.823	0.811	0.815	0.814	0.804	0.789	0.813	0.814	0.820	0.825	0.816
kc1	0.802	0.844	0.856	0.886	0.867	0.846	0.856	0.839	0.839	0.858	0.849	0.867
kc2	0.753	0.792	0.819	0.792	0.755	0.857	0.840	0.736	0.830	0.811	0.872	0.830
pc4	0.900	0.904	0.912	0.911	0.911	0.904	0.937	0.925	0.911	0.925	0.932	0.904
phoneme	0.824	0.874	0.804	0.832	0.756	0.861	0.889	0.824	0.778	0.867	0.889	0.872
qsar-biodeg	0.849	0.887	0.898	0.896	0.849	0.867	0.899	0.821	0.840	0.868	0.887	0.906
vehicle	0.772	0.824	0.802	0.788	0.859	0.747	0.837	0.859	0.800	0.765	0.882	0.859
AVERAGE	0.784	0.819	0.819	0.805	0.774	0.759	0.798	0.814	0.813	0.824	0.839	0.839

D Extended Operator Library

D.1 Added Operators

Table 8 lists the seven operators added on top of the 25-operator library of Table II in the main paper. Operator identifiers 0–24 are unchanged. All operators are implemented with scikit-learn primitives and default parameters unless stated otherwise; SMOTE is implemented directly on top of `NearestNeighbors` and introduces no additional dependency.

Table 8: Operators added to form the extended library ($N = 32$, $K = 7$). All are fitted on and applied to the training split only.

Category	Operator (ID)	Mechanism
Outlier Handling	Isolation Forest filter (25)	Removes training rows scored as anomalous by an isolation forest.
	LOF filter (26)	Removes training rows flagged as local density outliers.
	Winsorizer (27)	Clips each numeric column to the $[1\%, 99\%]$ quantile range of the training split.
	Z-score clipper (28)	Clips each numeric column to $\mu \pm 3\sigma$ of the training split.
Imbalance Handling	SMOTE (29)	Interpolates synthetic minority rows between a sample and its k same-class neighbours.
	Random under-sampler (30)	Down-samples majority classes toward the minority class size.
	Random over-sampler (31)	Duplicates minority rows toward the majority class size.

D.2 Operator-Level Constraints

In addition to the constraints of C.2, the extended library introduces the following, which prune sequences whose second operator cannot contribute:

- Two row-removing outlier operators (25, 26) may not be adjacent; the second one operates on an already filtered sample and carries almost no signal.
- Two clipping operators (27, 28) may not be adjacent, since the second acts on an already truncated distribution.
- At most one resampler (29–31) may appear in a pipeline, and two resamplers may not be adjacent; a later resampler overrides the class distribution produced by an earlier one.
- A row-removing outlier operator may not directly follow a resampler, which would discard the samples just synthesised.
- On datasets with at least 14 features, a resampler and a feature-expansion operator may not co-occur, since the product of synthesised rows and expanded columns dominates the evaluation cost.

D.3 Category Coherence under the Extended Taxonomy

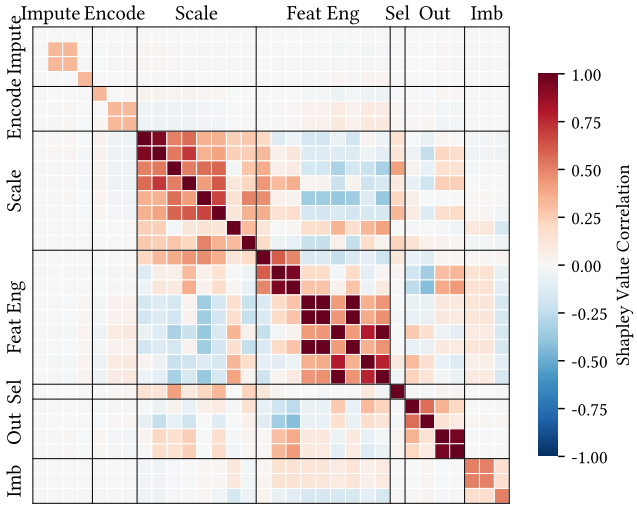


Figure 3: Pearson correlation coefficient of PipeShapley values across all positions for each pair of operators, on the extended library ($N = 32$, $K = 7$), computed as in Fig. 2 of the main paper. The block structure along the diagonal is preserved when two new functional families are added.

D.4 Per-Dataset Results

Table 9: Per-dataset accuracy on the extended operator library ($N = 32$, $K = 7$). Random, MC-SP and H-SP are averaged over the five seeds $(42, 114, 256, 512, 768)$**, with the across-seed standard deviation in parentheses. LOO is deterministic under a fixed train/test split and is reported from a single run. Δ is H-SP on the extended library minus H-SP on the original library (Table III in the main paper). "New ops" is the number of the five runs in which H-SP selected an operator from the two new categories (outlier handling, imbalance handling).**

Dataset	Random	LOO	MC-SP	H-SP	Δ	New ops
abalone	0.192 (0.069)	0.247	0.279 (0.004)	0.277 (0.004)	−0.001	1
ada_prior	0.806 (0.026)	0.807	0.841 (0.010)	0.849 (0.006)	+0.004	3
avila	0.467 (0.085)	0.756	0.764 (0.000)	0.943 (0.001)	+0.002	3
connect-4	0.591 (0.137)	0.793	0.795 (0.000)	0.793 (0.000)	−0.001	0
eeg	0.633 (0.119)	0.736	0.756 (0.000)	0.863 (0.006)	−0.001	2
google	0.561 (0.046)	0.671	0.674 (0.010)	0.672 (0.009)	+0.007	0
house_prices	0.773 (0.178)	0.911	0.934 (0.006)	0.923 (0.011)	−0.015	4
jungle_chess	0.703 (0.104)	0.866	0.864 (0.000)	0.864 (0.003)	+0.003	3
micro	0.443 (0.128)	0.633	0.636 (0.001)	0.640 (0.007)	−0.003	0
mozilla4	0.840 (0.035)	0.938	0.946 (0.005)	0.937 (0.004)	+0.003	0
obesity	0.708 (0.118)	0.882	0.886 (0.018)	0.914 (0.009)	+0.013	4
page-blocks	0.913 (0.014)	0.965	0.970 (0.002)	0.968 (0.008)	−0.002	2
pbcseq	0.686 (0.086)	0.715	0.777 (0.011)	0.769 (0.037)	−0.015	3
pol	0.823 (0.103)	0.981	0.964 (0.012)	0.975 (0.014)	−0.007	0
run_or_walk	0.747 (0.083)	0.972	0.973 (0.000)	0.972 (0.000)	−0.012	2
shuttle	0.898 (0.066)	1.000	0.999 (0.001)	1.000 (0.000)	±0.000	0
uscensus	0.668 (0.213)	0.846	0.846 (0.005)	0.846 (0.001)	±0.000	3
wall-robot	0.703 (0.169)	0.946	0.939 (0.005)	0.944 (0.006)	−0.008	4
cmc	0.551 (0.046)	0.588	0.601 (0.000)	0.593 (0.012)	−0.022	5
diabetes	0.800 (0.029)	0.844	0.886 (0.023)	0.870 (0.000)	±0.000	4
jm1	0.789 (0.045)	0.812	0.821 (0.003)	0.815 (0.000)	−0.009	0
kc1	0.700 (0.244)	0.863	0.878 (0.005)	0.869 (0.003)	−0.013	0
kc2	0.766 (0.028)	0.811	0.879 (0.010)	0.830 (0.000)	−0.038	1
pc4	0.845 (0.038)	0.884	0.933 (0.006)	0.926 (0.006)	−0.012	2
phoneme	0.716 (0.028)	0.867	0.871 (0.000)	0.875 (0.005)	−0.007	3
qsar-biodeg	0.734 (0.106)	0.868	0.902 (0.008)	0.887 (0.013)	−0.019	1
vehicle	0.633 (0.240)	0.906	0.885 (0.035)	0.918 (0.026)	+0.012	5
AVERAGE	0.692	0.819	0.833	0.842	−0.005	55/135

E Statistical Tests and Early-Termination Cases

E.1 Significance Tests

Table 10 compares H-SP with each baseline on the per-dataset mean accuracies (over five seeds) of Table III in the main paper, using a two-sided Wilcoxon signed-rank test with Holm correction across the 14 baselines, and reports the Friedman test over all methods that complete every dataset.

E.2 Early-Termination Cases

Table 11 lists representative datasets on which H-SP stops before the maximum length, together with the accuracy obtained when the pipeline is forced to length six. Disabling early termination decreases the average accuracy by one percentage point and roughly doubles runtime (Table IV in the main paper).

Table 10: H-SP versus each baseline on the 27 datasets of Table III in the main paper. W/T/L: datasets on which H-SP wins/ties/loses; Δ: mean accuracy difference (H-SP minus baseline, percentage points); p: two-sided Wilcoxon signed-rank test, Holm-corrected over the 14 baselines.

Baseline	W/T/L	Δ (pp)	p (Holm)
Rand	27/0/0	+8.47	7.3×10^{-5}
TPOT	24/2/1	+7.85	9.4×10^{-5}
AUSK	27/0/0	+10.93	7.3×10^{-5}
SAGA	26/0/1	+9.30	1.3×10^{-4}
DiffPrep	23/0/4	+6.34	1.5×10^{-4}
L2C	25/0/2	+4.03	7.3×10^{-5}
CtxPipe	22/2/3	+4.49	2.4×10^{-4}
HAIPipe	27/0/0	+7.30	7.3×10^{-5}
CatDB	26/0/1	+8.55	7.3×10^{-5}
ML-Agent†	23/0/1	+12.34	8.3×10^{-6}
GPT4o-ZS	27/0/0	+8.59	7.3×10^{-5}
GPT4o-FS	26/0/1	+5.59	7.3×10^{-5}
LOO	22/2/3	+2.61	3.5×10^{-4}
MC-SP	19/5/3	+1.14	1.4×10^{-3}
ES* (reference)	5/2/20	-1.49	$8.1\times10^{-4\ddagger}$

†Computed on the 24 datasets on which ML-Agent completes. ‡Uncorrected; ES* is a 10,000-evaluation reference, not a competing method. Friedman test over the 14 methods that complete all 27 datasets: $\chi^2_F = 151.0$, $p < 10^{-24}$.

Table 11: Impact of early termination on H-SP's accuracy. "Len. (w/. Early Term.)" denotes the pipeline length with early termination enabled; "Acc. (w/o Early Term.)" denotes performance when forced to use the maximum length $\overline{M} = 6$.

Dataset	Len.	Acc. (w/ early term.)	Acc. (w/o early term.)	ΔAcc
house_prices	6	0.938	0.938	0.0
obesity	2	0.901	0.882	-1.9
page-blocks	3	0.970	0.967	-0.3
cmc	3	0.615	0.601	-1.4
jm1	4	0.824	0.814	-1.0
kc1	3	0.882	0.877	-0.5
pc4	3	0.938	0.925	-1.3
qsar-biodeg	3	0.906	0.887	-1.9
Average (27 datasets)	3.2	0.847	0.837	-1.0